Experiment:4

Aim: Write a program to implement Breath First Search Traversal.

Source Code:

```
from collections import deque

class Graph:
    def __init__(self, vertices):

        self.V = vertices
        self.adj = [[] for _ in range(vertices)]

    def addEdge(self, v, w):

        self.adj[v].append(w)

    def BFS(self, s):
        # Mark all vertices as not visited
        visited = [False] * self.V

        # Create a queue for BFS
        queue = deque()

        # Mark starting vertex as visited and enqueue it
        visited[s] = True
        queue.append(s)

        result = []

        while queue:
            # Dequeue a vertex and print it
            s = queue.popleft()
            result.append(s)

            # Get all adjacent vertices of the dequeued vertex
            for i in self.adj[s]:
                if not visited[i]:
                    visited[i] = True
                    queue.append(i)

        return result


def main():
    """Main function to demonstrate BFS."""

    # Create a graph with 7 vertices (0 to 6)
    g = Graph(7)

    # Add edges (like in C++)
    g.addEdge(0, 1)
    g.addEdge(0, 2)
    g.addEdge(1, 3)
    g.addEdge(1, 4)
```

```python
    g.addEdge(2, 5)
    g.addEdge(2, 6)

    print("Breadth-First Search (BFS) starting from vertex 0:")

    # Perform BFS from vertex 0
    bfs_result = g.BFS(0)

    # Print result
    print("BFS Order:", " -> ".join(map(str, bfs_result)))


if __name__ == "__main__":
    main()
```

Output:

```
Breadth-First Search (BFS) starting from vertex 0:
BFS Order: 0 -> 1 -> 2 -> 3 -> 4 -> 5 -> 6
PS C:\Users\Student\Desktop\onkar_AI>
```

Experiment:5

Aim: Write a program to implement Water Jug Problem.

Source Code:

```
from collections import deque

def water_jug_bfs(c1, c2, target):
    queue = deque([((0, 0), [(0, 0)])])
    visited = {(0, 0)}

    while queue:
        (jug1, jug2), path = queue.popleft()


        if jug1 == target or jug2 == target:
            return path

        next_state = (c1, jug2)
        if next_state not in visited:
            visited.add(next_state)
            queue.append((next_state, path + [next_state]))

        next_state = (jug1, c2)
        if next_state not in visited:
            visited.add(next_state)
            queue.append((next_state, path + [next_state]))

        next_state = (0, jug2)
        if next_state not in visited:
            visited.add(next_state)
            queue.append((next_state, path + [next_state]))

        next_state = (jug1, 0)
        if next_state not in visited:
            visited.add(next_state)
            queue.append((next_state, path + [next_state]))

        pour_amount = min(jug1, c2 - jug2)
        next_state = (jug1 - pour_amount, jug2 + pour_amount)
        if next_state not in visited:
            visited.add(next_state)
            queue.append((next_state, path + [next_state]))

        pour_amount = min(jug2, c1 - jug1)
        next_state = (jug1 + pour_amount, jug2 - pour_amount)
        if next_state not in visited:
            visited.add(next_state)
            queue.append((next_state, path + [next_state]))

    return None


def print_solution(path, jug1_capacity, jug2_capacity):
    if path is None:
```

```python
        print("No solution exists!")
        return

    print(f"\nSolution found in {len(path) - 1} steps!")
    print(f"Jug1 Capacity: {jug1_capacity}L | Jug2 Capacity: {jug2_capacity}L")
    print("-" * 50)

    for i, (j1, j2) in enumerate(path):
        print(f"Step {i}: Jug1 = {j1}L, Jug2 = {j2}L")

    print("-" * 50)
    print(f"Final: Jug1={path[-1][0]}L, Jug2={path[-1][1]}L")


def main():

    print("=" * 50)
    print("       WATER JUG PROBLEM SOLVER")
    print("=" * 50)

    print("\n--- Example 1: 4L and 3L jugs, target = 2L ---")
    jug1_cap = 4
    jug2_cap = 3
    target = 2

    solution = water_jug_bfs(jug1_cap, jug2_cap, target)
    print_solution(solution, jug1_cap, jug2_cap)


if __name__ == "__main__":
    main()
```

Output:

```
==================================================
       WATER JUG PROBLEM SOLVER
==================================================

--- Example 1: 4L and 3L jugs, target = 2L ---

Solution found in 4 steps!
Jug1 Capacity: 4L | Jug2 Capacity: 3L
--------------------------------------------------
Step 0: Jug1 = 0L, Jug2 = 0L
Step 1: Jug1 = 0L, Jug2 = 3L
Step 2: Jug1 = 3L, Jug2 = 0L
Step 3: Jug1 = 3L, Jug2 = 3L
Step 4: Jug1 = 4L, Jug2 = 2L
--------------------------------------------------
Final: Jug1=4L, Jug2=2L
PS C:\Users\Student\Desktop\onkar_AI>
```